

```
In [15]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
plt.style.use('ggplot')
%matplotlib inline
```

```
In [16]: df = pd.read_csv(r"C:\Users\sneha\Downloads\Website_Traffic_Big_Dataset.csv")

df.head()
```

```
Out[16]:
```

|   | Date       | Page     | Traffic_Source | Sessions | Users | Bounce_Rate | Avg_Session_Duration_Min |
|---|------------|----------|----------------|----------|-------|-------------|--------------------------|
| 0 | 2025-01-01 | Blog     | Paid           | 3481     | 2531  | 67.05       | 3.74                     |
| 1 | 2025-01-02 | Blog     | Social         | 3488     | 3165  | 30.18       | 2.67                     |
| 2 | 2025-01-03 | Products | Referral       | 1294     | 995   | 45.17       | 5.64                     |
| 3 | 2025-01-04 | Pricing  | Direct         | 3873     | 3035  | 22.27       | 1.31                     |
| 4 | 2025-01-05 | Contact  | Paid           | 1807     | 1517  | 39.48       | 0.95                     |

```
In [18]: df.shape
```

```
Out[18]: (5000, 10)
```

```
In [20]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 10 columns):
#   Column              Non-Null Count  Dtype
---  -
0   Date                5000 non-null   object
1   Page                5000 non-null   object
2   Traffic_Source      5000 non-null   object
3   Sessions            5000 non-null   int64
4   Users               5000 non-null   int64
5   Bounce_Rate         5000 non-null   float64
6   Avg_Session_Duration_Min  5000 non-null   float64
7   Pages_Per_Session   5000 non-null   float64
8   Conversions         5000 non-null   int64
9   Goal_Completions    5000 non-null   int64
dtypes: float64(3), int64(4), object(3)
memory usage: 390.8+ KB
```

```
In [21]: df.describe()
```

```
Out[21]:
```

|       | Sessions    | Users       | Bounce_Rate | Avg_Session_Duration_Min | Pages_Per_Sessio |
|-------|-------------|-------------|-------------|--------------------------|------------------|
| count | 5000.000000 | 5000.000000 | 5000.000000 | 5000.000000              | 5000.000000      |
| mean  | 2572.467600 | 2120.537200 | 47.806956   | 4.204920                 | 4.55938          |
| std   | 1400.728032 | 1176.987616 | 15.949105   | 2.193558                 | 2.01916          |
| min   | 100.000000  | 78.000000   | 20.020000   | 0.500000                 | 1.00000          |
| 25%   | 1377.750000 | 1115.250000 | 33.690000   | 2.270000                 | 2.80750          |
| 50%   | 2567.000000 | 2100.500000 | 48.300000   | 4.190000                 | 4.59000          |
| 75%   | 3787.250000 | 3085.000000 | 61.842500   | 6.082500                 | 6.31000          |
| max   | 5000.000000 | 4675.000000 | 74.990000   | 7.990000                 | 8.00000          |



```
In [22]: df.isnull().sum()
```

```
Out[22]: Date                0
Page                0
Traffic_Source       0
Sessions            0
Users              0
Bounce_Rate         0
Avg_Session_Duration_Min  0
Pages_Per_Session   0
Conversions         0
Goal_Completions    0
dtype: int64
```

```
In [23]: df['Date'] = pd.to_datetime(df['Date'])

df.dtypes
```

```
Out[23]: Date                datetime64[ns]
Page                object
Traffic_Source       object
Sessions            int64
Users              int64
Bounce_Rate         float64
Avg_Session_Duration_Min  float64
Pages_Per_Session   float64
Conversions         int64
Goal_Completions    int64
dtype: object
```

## Total Sessions

```
In [24]: total_sessions = df['Sessions'].sum()

print("Total Sessions:", total_sessions)
```

Total Sessions: 12862338

## Total Users

```
In [25]: total_users = df['Users'].sum()

print("Total Users:", total_users)
```

Total Users: 10602686

## Total Conversions

```
In [26]: total_conversions = df['Conversions'].sum()

print("Total Conversions:", total_conversions)
```

Total Conversions: 770378

## Bounce Rate Analysis

```
In [27]: avg_bounce_rate = df['Bounce_Rate'].mean()

print("Average Bounce Rate:", round(avg_bounce_rate,2))
```

Average Bounce Rate: 47.81

## Average Session Duration

```
In [28]: avg_duration = df['Avg_Session_Duration_Min'].mean()

print("Average Session Duration:", round(avg_duration,2))
```

Average Session Duration: 4.2

## Average Pages Per Session

```
In [29]: avg_pages = df['Pages_Per_Session'].mean()

print("Average Pages Per Session:", round(avg_pages,2))
```

Average Pages Per Session: 4.56

# Traffic Source Analysis

```
In [31]: traffic_source = df.groupby('Traffic_Source')['Sessions'].sum().reset_index()

traffic_source
```

```
Out[31]:
```

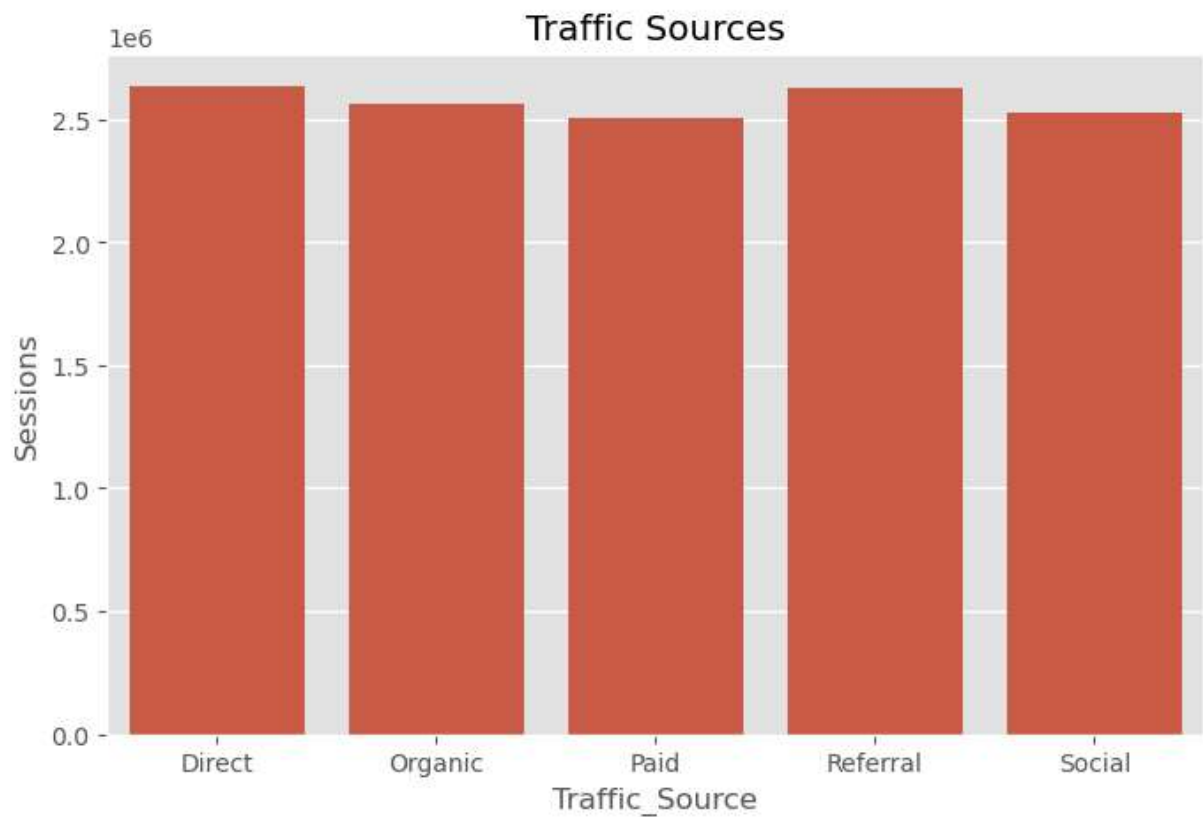
|   | Traffic_Source | Sessions |
|---|----------------|----------|
| 0 | Direct         | 2635350  |
| 1 | Organic        | 2562801  |
| 2 | Paid           | 2507646  |
| 3 | Referral       | 2626987  |
| 4 | Social         | 2529554  |

## Top Traffic Sources Chart

```
In [32]: plt.figure(figsize=(8,5))

sns.barplot(
    x='Traffic_Source',
    y='Sessions',
    data=traffic_source
)

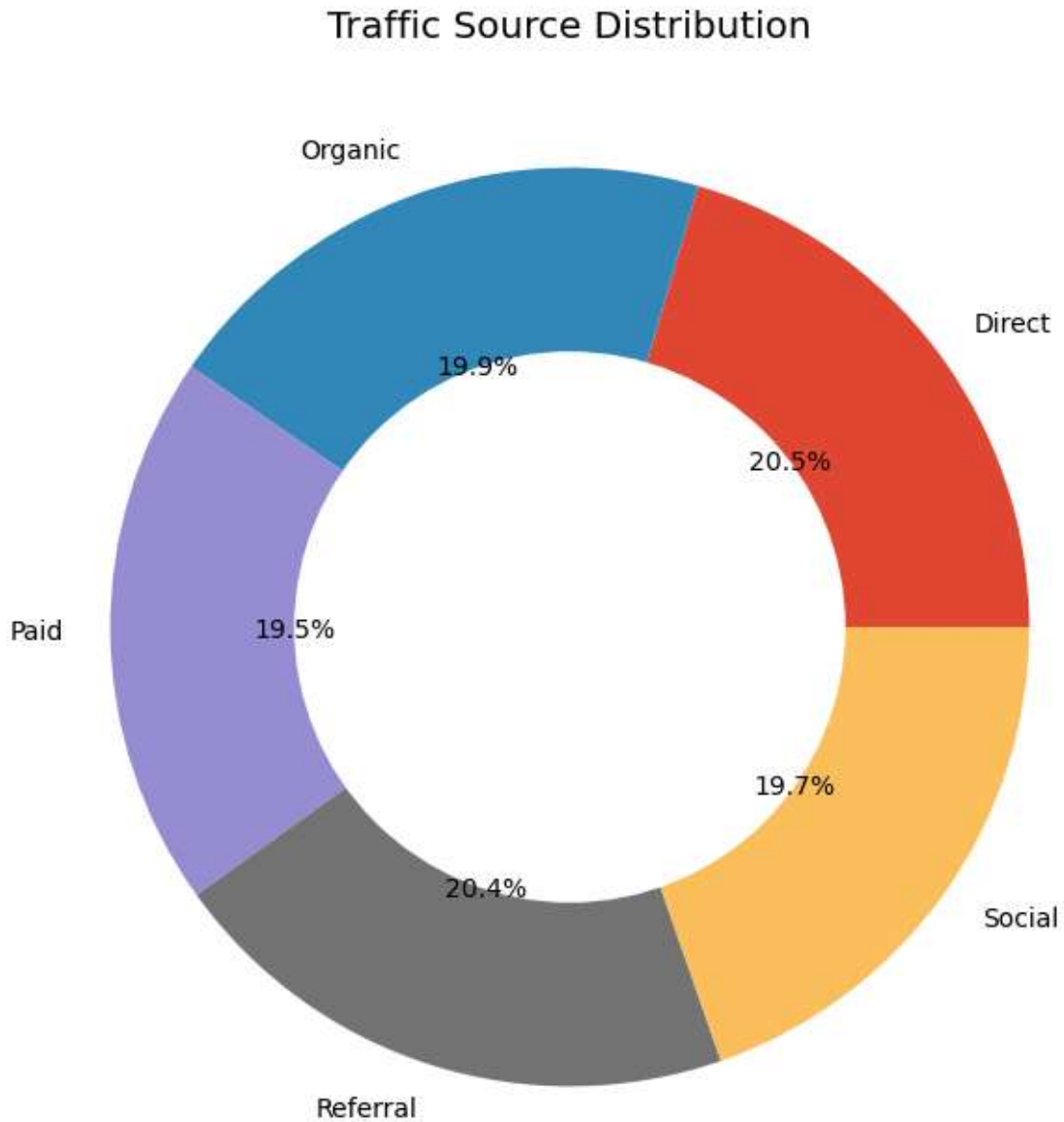
plt.title("Traffic Sources")
plt.show()
```



## Donut Chart

```
In [33]: traffic_source = df.groupby(  
        'Traffic_Source'  
    )['Sessions'].sum()  
  
    plt.figure(figsize=(8,8))  
  
    plt.pie(  
        traffic_source,  
        labels=traffic_source.index,  
        autopct='%1.1f%%'  
    )  
  
    centre_circle = plt.Circle(  
        (0,0),  
        0.60,  
        fc='white'  
    )  
  
    fig = plt.gcf()  
  
    fig.gca().add_artist(  
        centre_circle  
    )  
  
    plt.title("Traffic Source Distribution")
```

```
plt.show()
```



## Sessions Trend

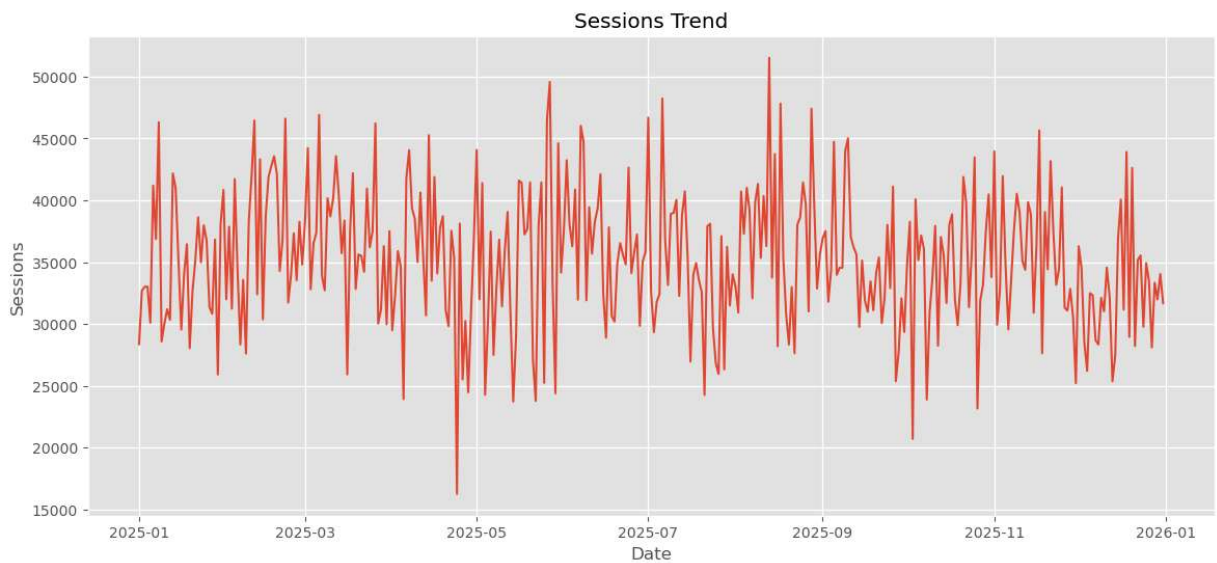
```
In [35]: daily_sessions = df.groupby(  
        'Date'  
        )['Sessions'].sum()
```

```
In [36]: plt.figure(figsize=(14,6))  
  
plt.plot(  
    daily_sessions.index,  
    daily_sessions.values  
)
```

```
plt.title("Sessions Trend")

plt.xlabel("Date")
plt.ylabel("Sessions")

plt.show()
```



## Area Chart

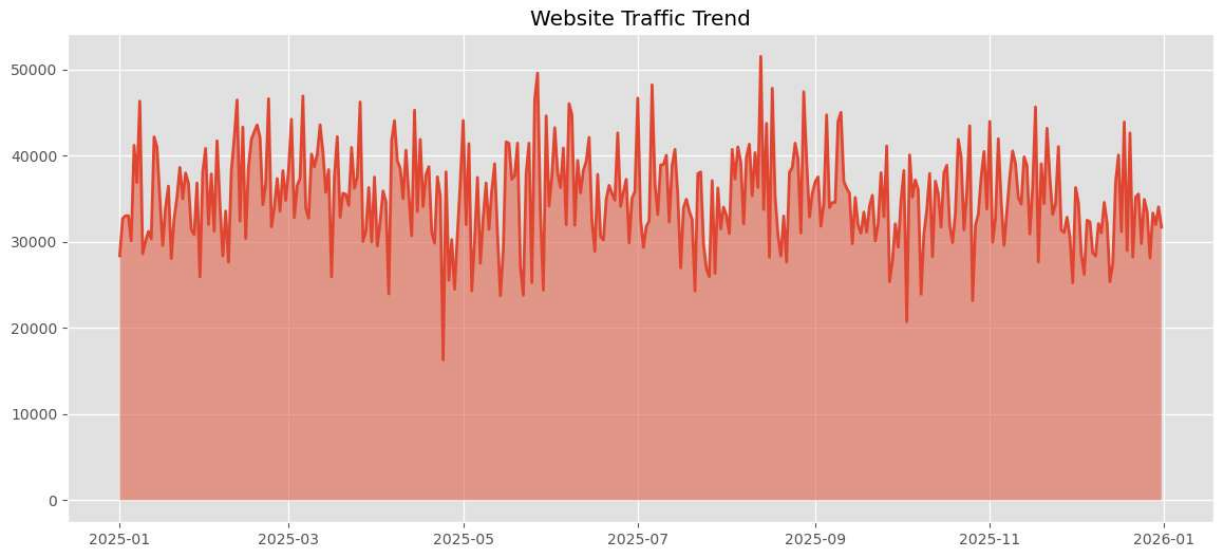
```
In [37]: plt.figure(figsize=(14,6))

plt.fill_between(
    daily_sessions.index,
    daily_sessions.values,
    alpha=0.5
)

plt.plot(
    daily_sessions.index,
    daily_sessions.values,
    linewidth=2
)

plt.title("Website Traffic Trend")

plt.show()
```



## Conversion Analysis

### Conversion Rate

```
In [38]: conversion_rate = (  
    df['Conversions'].sum()  
    /  
    df['Sessions'].sum()  
    ) * 100  
  
print(  
    round(conversion_rate,2),  
    "%"  
    )
```

5.99 %

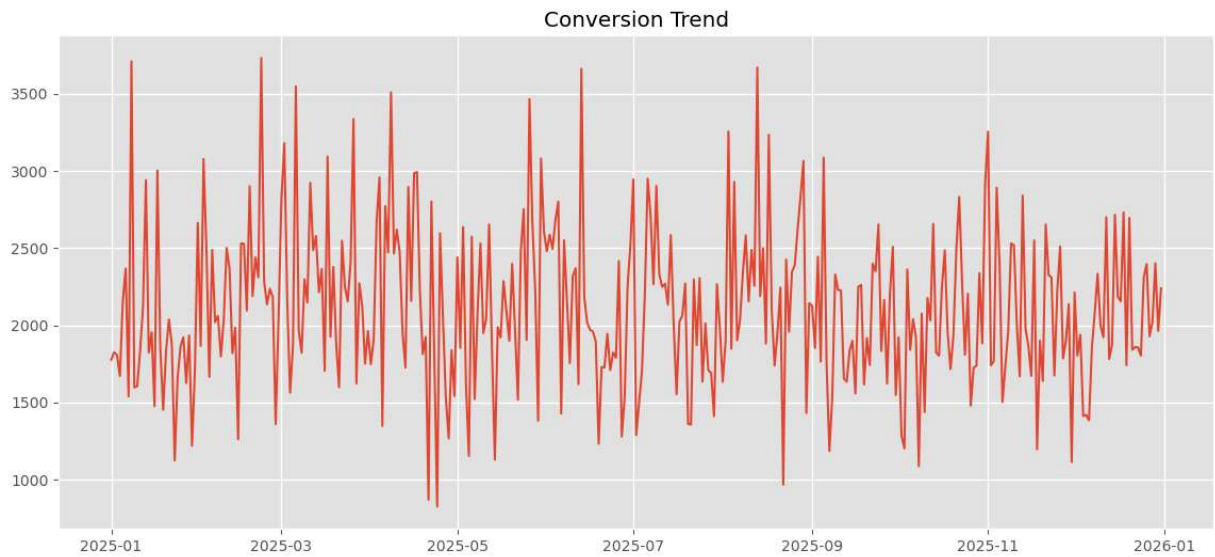
### Conversion Trend

```
In [40]: daily_conversion = df.groupby(  
    'Date'  
    )['Conversions'].sum()
```

```
In [41]: plt.figure(figsize=(14,6))  
  
plt.plot(  
    daily_conversion.index,  
    daily_conversion.values  
    )  
  
plt.title(  
    "Conversion Trend"  
    )
```



```
plt.show()
```



## Goal Completion Analysis

```
In [42]: goal_rate = (  
    df['Goal Completions'].sum()  
    /  
    df['Sessions'].sum()  
    ) * 100  
  
print(  
    round(goal_rate, 2),  
    "%"  
)
```

5.98 %

## High Performing Pages

```
In [43]: top_pages = df.groupby(  
    'Page'  
    )['Conversions'].sum()  
  
top_pages = top_pages.sort_values(  
    ascending=False  
    )  
  
top_pages
```

```
Out[43]: Page
Landing Page    102799
Blog            101469
Pricing         99707
Home           97965
Contact         96108
About          95130
Services        92607
Products        84593
Name: Conversions, dtype: int64
```

## Top Pages Chart

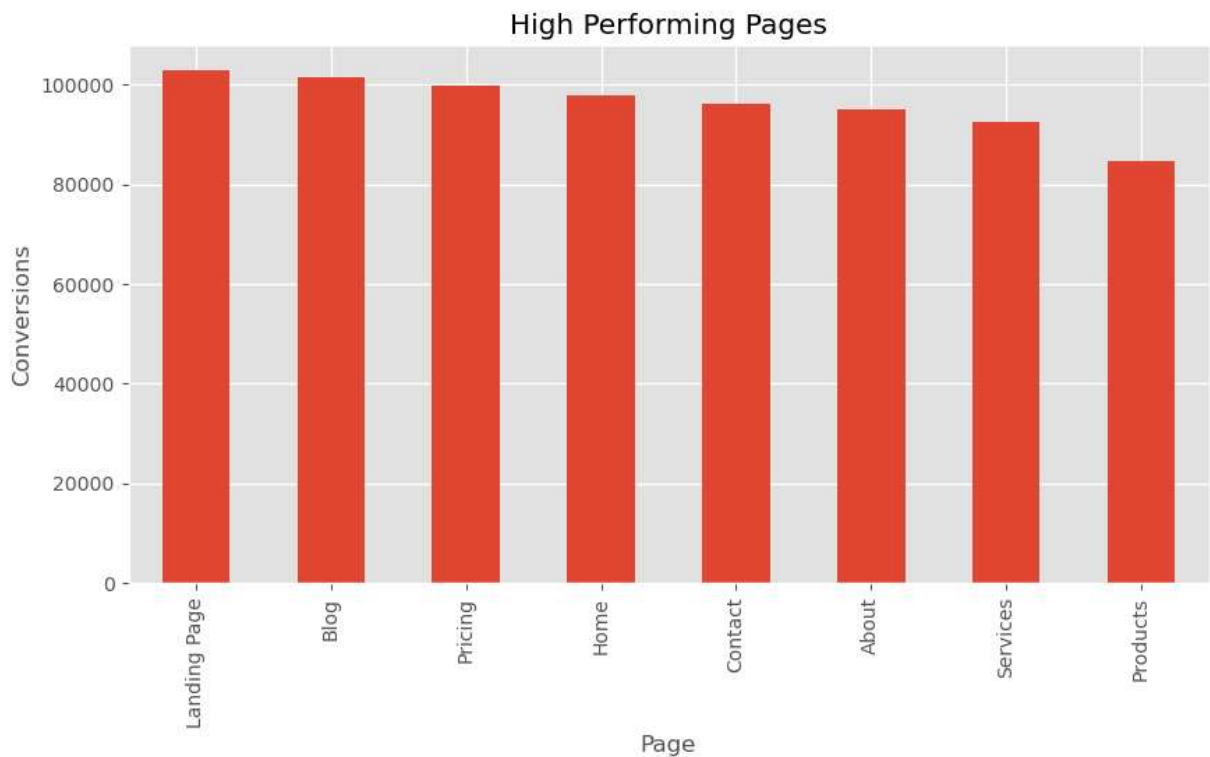
```
In [44]: plt.figure(figsize=(10,5))

top_pages.plot(
    kind='bar'
)

plt.title(
    "High Performing Pages"
)

plt.ylabel(
    "Conversions"
)

plt.show()
```



# Low Performing Pages

```
In [45]: low_pages = df.groupby(
          'Page'
        )['Bounce_Rate'].mean()

low_pages = low_pages.sort_values(
    ascending=False
)

low_pages
```

```
Out[45]: Page
Blog      48.694335
Pricing   48.662516
Home      48.561990
Services  48.195700
About     47.931664
Landing Page 47.875093
Contact   46.440303
Products  45.939828
Name: Bounce_Rate, dtype: float64
```

## Bounce Rate Chart

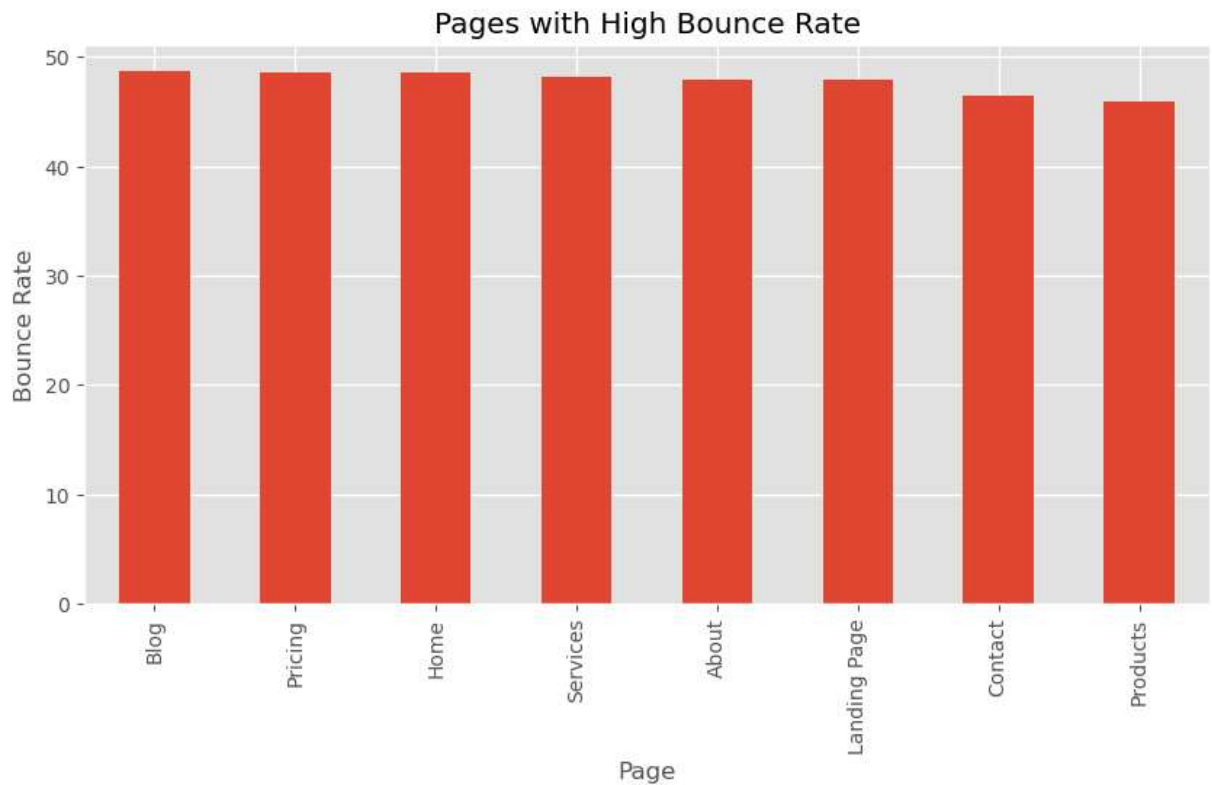
```
In [46]: plt.figure(figsize=(10,5))

low_pages.plot(
    kind='bar'
)

plt.title(
    "Pages with High Bounce Rate"
)

plt.ylabel(
    "Bounce Rate"
)

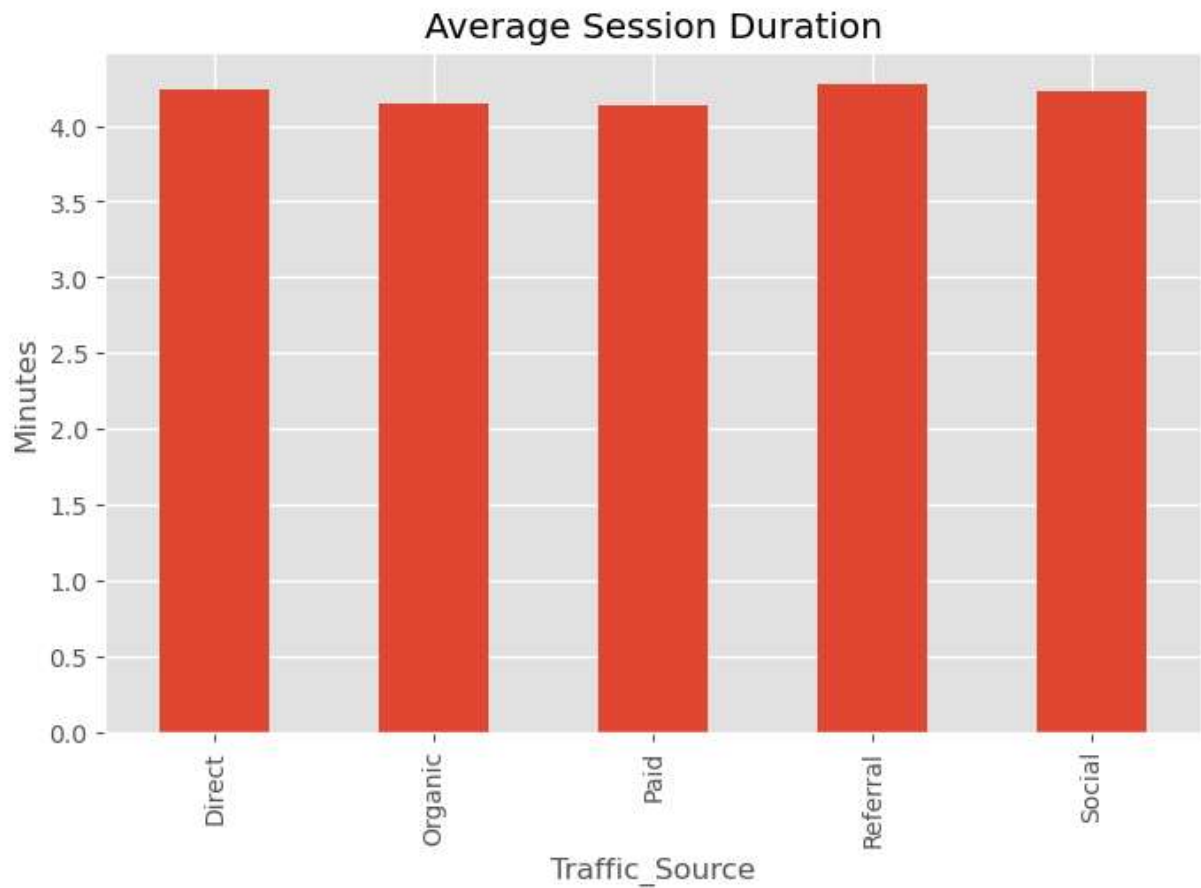
plt.show()
```



## Session Duration Analysis

```
In [48]: engagement = df.groupby(  
        'Traffic_Source'  
    ) [  
        'Avg_Session_Duration_Min'  
    ].mean()
```

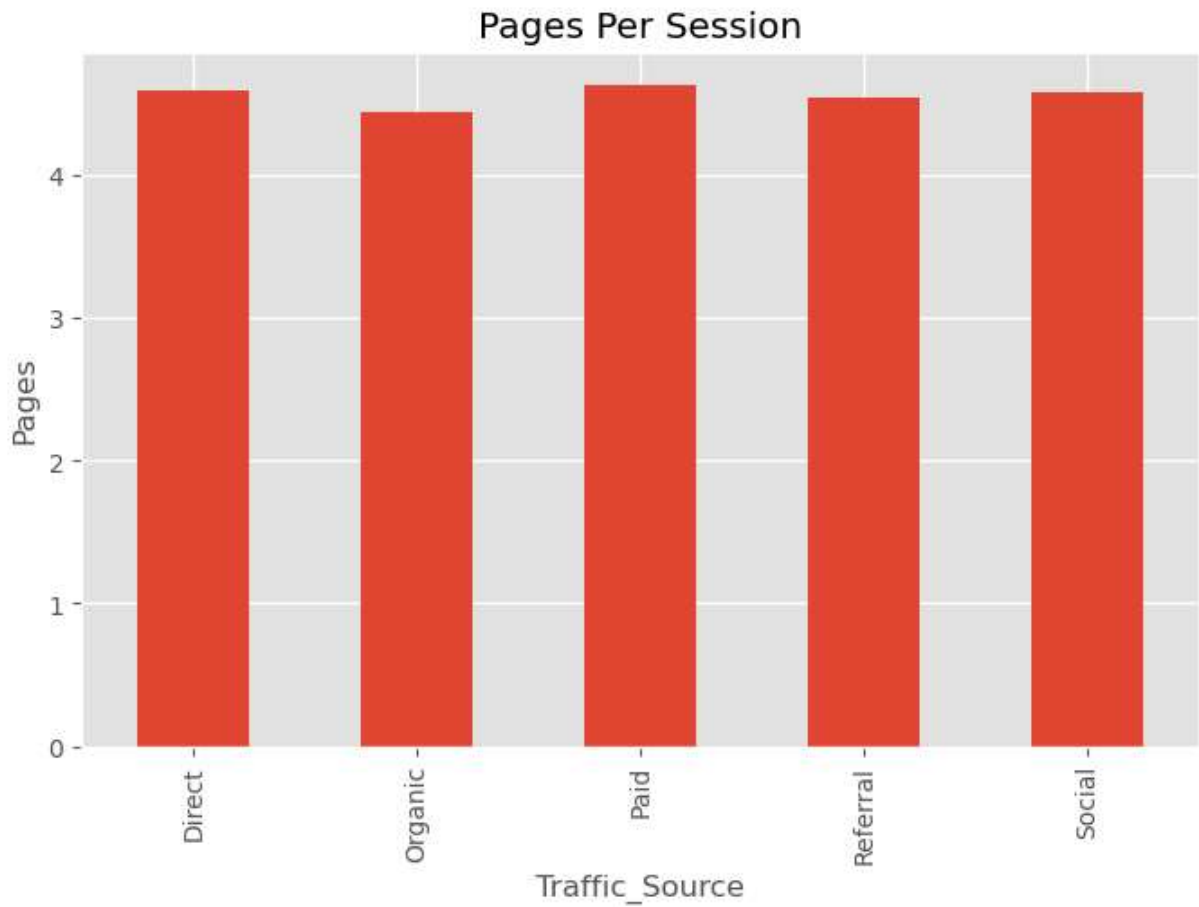
```
In [50]: plt.figure(figsize=(8,5))  
  
engagement.plot(  
    kind='bar'  
)  
  
plt.title(  
    "Average Session Duration"  
)  
  
plt.ylabel(  
    "Minutes"  
)  
  
plt.show()
```



## Pages Per Session

```
In [51]: pages_session = df.groupby(  
        'Traffic_Source'  
    )[  
        'Pages_Per_Session'  
    ].mean()
```

```
In [52]: plt.figure(figsize=(8,5))  
  
pages_session.plot(  
    kind='bar'  
)  
  
plt.title(  
    "Pages Per Session"  
)  
  
plt.ylabel(  
    "Pages"  
)  
  
plt.show()
```



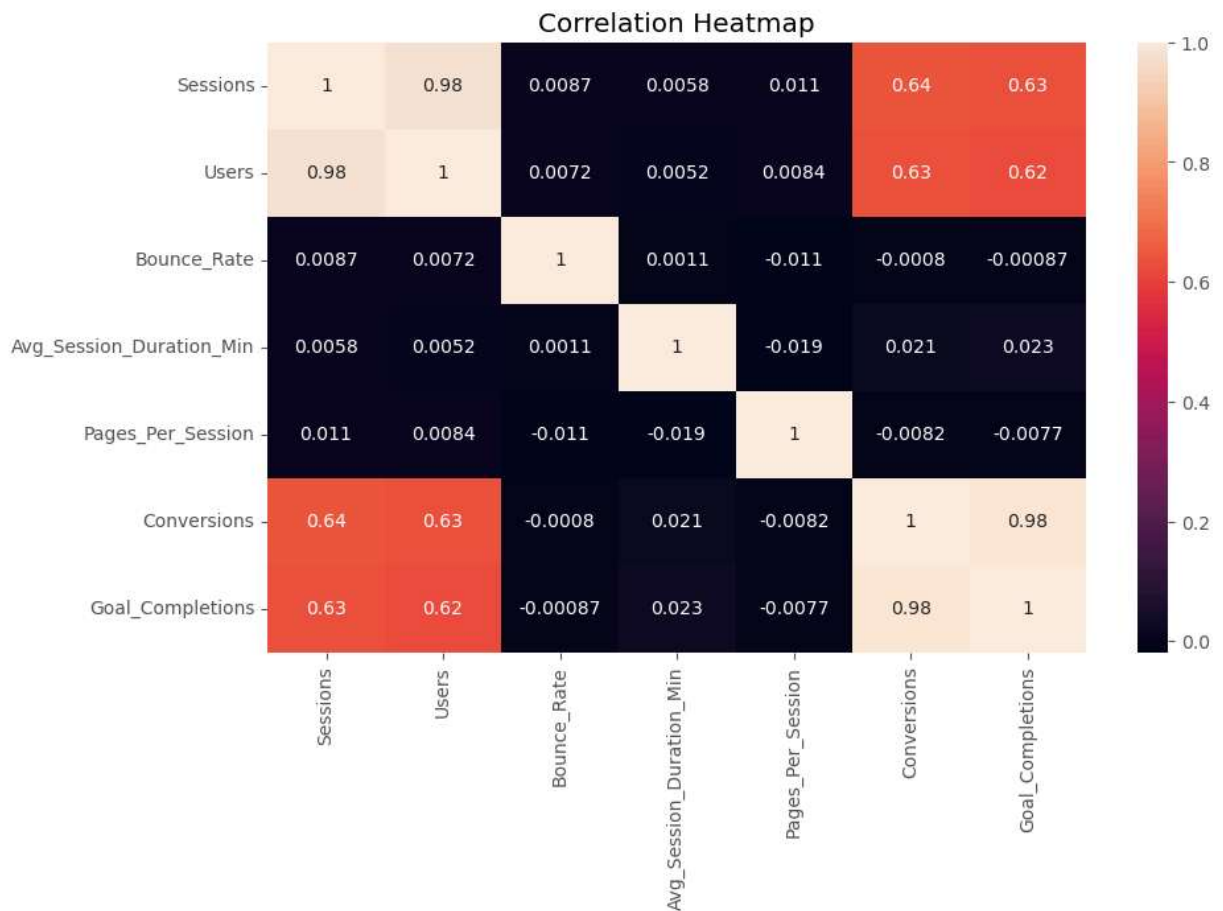
## Correlation Heatmap

```
In [53]: plt.figure(figsize=(10,6))

sns.heatmap(
    df.select_dtypes(
        include=np.number
    ).corr(),
    annot=True
)

plt.title(
    "Correlation Heatmap"
)

plt.show()
```



## Extract Business Insights

### Top Traffic Source

```
In [54]: df.groupby('Traffic_Source')['Sessions'].sum().sort_values(ascending=False)
```

```
Out[54]: Traffic_Source
Direct      2635350
Referral    2626987
Organic     2562801
Social      2529554
Paid        2507646
Name: Sessions, dtype: int64
```

### Best Converting Traffic Source

```
In [55]: df.groupby('Traffic_Source')['Conversions'].sum().sort_values(ascending=False)
```

```
Out[55]: Traffic_Source
Direct      161982
Referral    159019
Social      153433
Organic     152006
Paid        143938
Name: Conversions, dtype: int64
```

## Highest Bounce Rate Source

```
In [56]: df.groupby('Traffic_Source')['Bounce_Rate'].mean().sort_values(ascending=False)
```

```
Out[56]: Traffic_Source
Referral    48.581464
Paid        48.285889
Direct      47.918413
Organic     47.234458
Social      46.974635
Name: Bounce_Rate, dtype: float64
```

## Identify Top Performing Pages

```
In [57]: top_pages = df.groupby('Page')['Conversions'].sum().sort_values(ascending=False)
print(top_pages.head())
```

```
Page
Landing Page    102799
Blog             101469
Pricing          99707
Home             97965
Contact          96108
Name: Conversions, dtype: int64
```

## Identify Low Performing Pages

```
In [58]: low_pages = df.groupby('Page')['Bounce_Rate'].mean().sort_values(ascending=False)
print(low_pages.head())
```

```
Page
Blog           48.694335
Pricing         48.662516
Home            48.561990
Services        48.195700
About           47.931664
Name: Bounce_Rate, dtype: float64
```

## Total Sessions



```
In [61]: print(df['Sessions'].sum())
```

12862338

## Total Users

```
In [62]: print(df['Users'].sum())
```

10602686

## Total Conversions

```
In [63]: print(df['Conversions'].sum())
```

770378

## Conversion Rate

```
In [64]: conversion_rate = (df['Conversions'].sum() / df['Sessions'].sum()) * 100  
print(round(conversion_rate,2))
```

5.99

## Average Bounce Rate

```
In [65]: print(round(df['Bounce_Rate'].mean(),2))
```

47.81

## EXECUTIVE SUMMARY

- Total Sessions: 12,862,338
- Total Users: 10,602,686
- Total Conversions: 77,0378
- Conversion Rate: 5.99%
- Average Bounce Rate: 47.8%

```
In [ ]:
```